

Prompt:

8/17/2026, 9:44:08 PM

lanjut Tahap C

Response:

8/17/2026, 9:44:08 PM

Tentu. **Tahap C** adalah titik di mana proyek kita mulai benar-benar berubah dari model forecasting menjadi sistem quantitative trading.

Tahap A memberi kita **state pasar**:

$$S_t$$

Tahap B memberi kita **forecast distribusi return**:

$$\mu_t, \Sigma_t, B_t$$

Sekarang Tahap C harus menjawab:

Pada kondisi pasar S_t , aset mana yang sebaiknya diperdagangkan, dalam arah apa, dengan ukuran posisi berapa, pada harga entry/SL/TP seperti apa, sehingga expected utility maksimum dengan batas risiko yang telah ditentukan?

Dan di sinilah **TSP, Max-Cut, portfolio optimization, QUBO, Ising, dan akhirnya QAOA** mulai masuk secara natural.

C. Tujuan Tahap C

Kita ingin mengubah:

$$(\mu_t, \Sigma_t, B_t)$$

menjadi:

$$\mathbf{x}_t^*$$

dengan $\mathbf{x}_t^*$ adalah keputusan trading optimal.

Secara abstrak:

$$\mathbf{x}_t^* = \arg \max_{\mathbf{x} \in X} U(\mathbf{x} \mid S_t)$$

dengan:

- $\mathbf{x}$: keputusan trading,
- X : himpunan keputusan yang diperbolehkan,
- U : objective/utility.

C.1. Jangan langsung berpikir "portfolio"

Untuk sistem kita, decision variable sebenarnya lebih kaya.

Untuk setiap aset i , kita ingin mengetahui:

$$x_i = \text{posisi trading aset } i$$

Tetapi:

$$x_i$$

tidak cukup.

Karena kita perlu membedakan:

LONG

SHORT

FLAT.

Maka secara konseptual:

$$x_i \in \{-1, 0, +1\}.$$

Yaitu:

$$-1 = \text{short}$$

$$0 = \text{flat}$$

$$+1 = \text{long.}$$

Ini sudah merupakan **discrete optimization problem**.

Dan ini penting karena quantum optimization sangat cocok dengan decision variable diskrit.

C.2. Tetapi position size juga harus diputuskan

Misalkan:

$$x_i = 1.$$

Itu hanya mengatakan:

"Trade long."

Belum mengatakan berapa besar posisi.

Kita definisikan:

$$w_i \geq 0$$

sebagai position size/notional.

Maka:

$$p_i = x_i w_i.$$

Jika:

$$x_i = +1,$$

maka:

$$p_i = +w_i.$$

Jika:

$$x_i = -1,$$

maka:

$$p_i = -w_i.$$

Jika:

$$x_i = 0,$$

maka:

$$p_i = 0.$$

C.3. Namun kita mempunyai constraint yang sangat spesifik

Sebelumnya kamu menentukan:

setiap trade harus memiliki risiko \$1.

Ini sangat penting.

Misalkan:

$$P_i$$

adalah entry price.

Stop loss:

$$SL_i.$$

Risk per unit:

$$d_i = |P_i - SL_i|.$$

Jika posisi berukuran:

$$q_i$$

unit, maka dollar risk:

$$R_i = q_i d_i.$$

Kita ingin:

$$R_i \leq R_{\max}$$

dengan:

$$R_{\max} = \$1.$$

Jika ingin tepat \$1:

$$q_i = \frac{1}{|P_i - SL_i|}$$

untuk aset dengan satuan harga yang sesuai.

C.4. Ini menghasilkan hubungan penting

Position size bukan dipilih sembarangan.

Ia berasal dari:

$$\text{Risk Budget} \div \text{Stop Distance}$$

atau:

$$q_i = \frac{R_{\max}}{|P_i - SL_i|}$$

Contoh:

Entry:

$$P_i = 100$$

SL:

$$SL_i = 99.$$

Maka:

$$d_i = 1.$$

Jika:

$$R_{\max} = \$1,$$

maka:

$$q_i = 1.$$

Tetapi jika:

$$SL_i = 98,$$

maka:

$$d_i = 2$$

dan:

$$q_i = 0.5.$$

Jadi:

wider SL $\Rightarrow$ smaller position.

C.5. Mengapa ini berbeda dari portfolio optimization biasa?

Markowitz biasanya berbicara tentang:

$$w_i$$

sebagai bobot modal.

Sistem kita lebih dekat ke:

risk-normalized position.

Kita tidak mengatakan:

"Saya mau mengalokasikan 10% modal."

Kita mengatakan:

"Jika trade ini gagal dan SL tersentuh, kerugiannya maksimal \$1."

Ini jauh lebih cocok untuk short-term systematic trading.

C.6. Sekarang kita masukkan forecast

Tahap B memberi:

$$\mu_i = E[R_i | S_i].$$

Expected PnL jika posisi p_i :

$$E[P_i] = p_i \mu_i.$$

Untuk seluruh portfolio:

$$E[P] = \mathbf{p}^T \boldsymbol{\mu}.$$

Jika:

$$\mathbf{p} = [p_1, \dots, p_N]^T,$$

maka:

$$E[P] = \sum_i p_i \mu_i.$$

C.7. Sekarang masuk covariance

Variance portfolio:

$$\sigma_P^2 = \mathbf{p}^T \Sigma \mathbf{p}.$$

Ekspansi:

$$\sigma_p^2 = \sum_i \sum_j p_i p_j \Sigma_{ij}.$$

Perhatikan:

$$\Sigma_{ij}$$

menciptakan interaksi antara trade i dan j .

Ini sangat penting.

Misalnya kita memiliki dua aset:

BTC

dan:

ETH.

Jika:

$$\rho_{BTC,ETH} \approx 1,$$

membuka long BTC dan long ETH bukan dua independent bets.

Mereka hampir merupakan:

one common risk exposure

C.8. Maka objective dasar kita

Kita dapat menggunakan mean-variance utility:

$$U(\mathbf{p}) = \boldsymbol{\mu}^T \mathbf{p} - \lambda \mathbf{p}^T \Sigma \mathbf{p}.$$

Dengan:

$$\lambda > 0$$

sebagai risk aversion.

Ini mirip Markowitz, tetapi sekarang:

$\mathbf{p}$

adalah **trading positions**, bukan sekadar strategic portfolio weights.

C.9. Tetapi kita mempunyai factor exposure

Tahap B juga memberikan:

B .

Portfolio factor exposure:

$$\mathbf{e}_F = B^T \mathbf{p}.$$

Misalnya factor:

$$F = [BTC, ETH, USD, GOLD, SPX].$$

Maka:

$$B^T \mathbf{p}$$

memberitahu kita berapa besar portfolio sebenarnya bertaruh terhadap faktor-faktor tersebut.

C.10. Ini sangat penting untuk contoh altcoin

Misalkan kita membuka:

$LONG(SOL)$

dan:

$LONG(AVAX)$.

Mungkin kita berpikir:

"Saya punya dua trade."

Tetapi factor model mengatakan:

$$\beta_{\text{SOL},\text{BTC}} = 1.3$$

dan:

$$\beta_{\text{AVAX},\text{BTC}} = 1.5.$$

Maka:

$$e_{\text{BTC}} = 1.3p_{\text{SOL}} + 1.5p_{\text{AVAX}}.$$

Jadi portfolio kita sebenarnya adalah:

large BTC directional bet

meskipun kita tidak membeli BTC.

C.11. Maka kita bisa memberikan constraint

Misalnya:

$$|e_{\text{BTC}}| \leq E_{\text{BTC}}^{\max}.$$

Secara matematis:

$$|B_{\text{BTC}}^T \mathbf{p}| \leq E_{\text{BTC}}^{\max}.$$

Begitu juga:

$$|e_{\text{USD}}| \leq E_{\text{USD}}^{\max}.$$

dan:

$$|e_{\text{Gold}}| \leq E_{\text{Gold}}^{\max}.$$

C.12. Ini membuat sistem kita jauh lebih menarik

Karena sekarang optimizer tidak hanya bertanya:

"Trade mana yang menghasilkan profit terbesar?"

Tetapi:

"Trade mana yang memberikan profit terbesar **setelah memperhitungkan covariance dan factor exposure?**"

Itu jauh lebih dekat dengan quantitative portfolio construction.

C.13. Sekarang kita masuk ke entry

Forecast μ_i sebenarnya belum cukup.

Kita perlu menentukan:

$$P_i^{entry}.$$

Ada beberapa kemungkinan:

Market entry

$$P_i^{entry} = P_{market}.$$

Limit entry

$$P_i^{entry} = P_{market} - \delta_i$$

untuk long.

Atau:

$$P_i^{entry} = P_{market} + \delta_i$$

untuk short.

Jadi entry distance:

$$\boxed{\delta_i}$$

sendiri menjadi decision variable.

C.14. Sekarang TP dan SL

Untuk long:

$$SL_i = P_i - \Delta_i^{SL}$$

dan:

$$TP_i = P_i + \Delta_i^{TP}.$$

Untuk short:

$$SL_i = P_i + \Delta_i^{SL}$$

dan:

$$TP_i = P_i - \Delta_i^{TP}.$$

Maka kita mempunyai dua parameter:

$$\boxed{d_i^{SL}}$$

dan:

$$\boxed{d_i^{TP}.$$

C.15. Risk-reward ratio

Definisikan:

$$RR_i = \frac{d_i^{TP}}{d_i^{SL}}.$$

Misalnya:

$$d^{SL} = 1$$

dan:

$$d^{TP} = 2.$$

Maka:

$$RR = 2.$$

C.16. Tetapi RR tinggi tidak otomatis bagus

Misalnya:

$$RR = 3.$$

Tetapi:

$$P(TP) = 0.20.$$

Expected value:

$$EV = 0.20(3) - 0.80(1).$$

Sehingga:

$$EV = -0.20.$$

Sebaliknya:

$$RR = 1.5$$

dengan:

$$P(TP) = 0.60$$

memberikan:

$$EV = 0.60(1.5) - 0.40(1)$$

$$= 0.5.$$

Jadi optimizer kita harus mempertimbangkan:

$$P(TP), \quad P(SL), \quad RR.$$

C.17. Expected trade value

Misalkan:

$$G_i$$

adalah keuntungan jika TP tercapai.

Dan:

$$L_i$$

adalah kerugian jika SL tercapai.

Maka:

$$EV_i = P_i^{TP} G_i - P_i^{SL} L_i - C_i.$$

Jika kita memaksa:

$$L_i = \$1,$$

maka:

$$EV_i = P_i^{TP} G_i - P_i^{SL} - C_i.$$

Jika hanya ada TP/SL:

$$P_i^{SL} = 1 - P_i^{TP},$$

maka:

$$EV_i = P_i^{TP} G_i - (1 - P_i^{TP}) - C_i.$$

C.18. Sekarang kita mempunyai dua level objective

Level individual trade

$$EV_i.$$

Level portfolio

$$U(\mathbf{p}) = \boldsymbol{\mu}^T \mathbf{p} - \lambda \mathbf{p}^T \Sigma \mathbf{p}.$$

Kita tidak ingin memilih trade hanya berdasarkan:

$$EV_i.$$

Karena dua trade dengan EV tinggi dapat sangat berkorelasi.

Maka objective akhir harus menggabungkan keduanya.

C.19. Salah satu formulasi awal yang sangat bagus

Definisikan expected net return vector:

$$\tilde{\boldsymbol{\mu}} = \begin{bmatrix} EV_1 \\ EV_2 \\ \vdots \\ EV_N \end{bmatrix}.$$

Kemudian:

$$U(\mathbf{p}) = \tilde{\boldsymbol{\mu}}^T \mathbf{p} - \lambda \mathbf{p}^T \Sigma \mathbf{p}.$$

Ini adalah:

$$\text{expected trading value} - \text{portfolio risk penalty}.$$

C.20. Sekarang masalah kita sudah menjadi optimization problem

$$\max_{\mathbf{p}} \quad \tilde{\boldsymbol{\mu}}^T \mathbf{p} - \lambda \mathbf{p}^T \Sigma \mathbf{p}$$

subject to:

$$|p_i| \leq p_i^{\max}$$

$$\sum_i |p_i| \leq L_{\max}$$

$$|\mathbf{B}^T \mathbf{p}| \leq \mathbf{e}_{\max}.$$

Dan:

$$\text{risk budget} \leq R_{\max}.$$

C.21. Tetapi kita belum mempunyai binary optimization

Karena:

$$p_i$$

masih continuous.

Untuk masuk ke QUBO/Ising, kita perlu membuat keputusan diskrit.

Saya ingin membangun ini **secara bertahap**, bukan memaksakan continuous optimization menjadi Ising.

C.22. Kita mulai dengan decision variable binary

Untuk setiap aset:

$$y_i^L \in \{0, 1\}$$

dan:

$$y_i^S \in \{0, 1\}.$$

Interpretasi:

$$y_i^L = 1 \Rightarrow \text{long}$$

$$y_i^S = 1 \Rightarrow \text{short.}$$

Kita harus mencegah:

$$y_i^L = y_i^S = 1.$$

Constraint:

$$y_i^L + y_i^S \leq 1.$$

C.23. Posisi signed

Definisikan:

$$x_i = y_i^L - y_i^S.$$

Maka:

$$x_i \in \{-1, 0, +1\}.$$

y_i^L	y_i^S	x_i
0	0	0
1	0	+1
0	1	-1
1	1	forbidden

C.24. Ini sudah sangat dekat dengan QUBO

Kita punya:

$$x_i = y_i^L - y_i^S.$$

Expected value:

$$\tilde{\mu}_i x_i.$$

Risk:

$$x^T Q x$$

dengan:

$$Q \sim \Sigma.$$

Jadi objective:

$$U(y) = \sum_i \tilde{\mu}_i (y_i^L - y_i^S) - \lambda \sum_{ij} (y_i^L - y_i^S) \Sigma_{ij} (y_j^L - y_j^S).$$

C.25. Constraint long-short

Kita tambahkan penalty:

$$P_{LS} \sum_i y_i^L y_i^S.$$

Karena jika:

$$y_i^L = y_i^S = 1,$$

maka:

$$y_i^L y_i^S = 1.$$

Sehingga objective menjadi lebih buruk.

Jadi:

$$U' = U - P_{LS} \sum_i y_i^L y_i^S.$$

Jika:

$$P_{LS} \gg \text{maximum possible benefit,}$$

solusi optimal tidak akan memilih keduanya.

C.26. Constraint jumlah trade

Misalkan maksimum:

$$K$$

trade sekaligus.

Kita ingin:

$$\sum_i (y_i^L + y_i^S) \leq K.$$

Untuk QUBO, constraint inequality dapat diubah menggunakan penalty.

Misalnya:

$$(\sum_i (y_i^L + y_i^S) - K)^2.$$

Kemudian:

$$P_K (\sum_i (y_i^L + y_i^S) - K)^2$$

ditambahkan ke objective sebagai penalty.

C.27. Constraint factor exposure

Misalnya kita ingin:

$$B_k^T x = 0$$

untuk market-neutral terhadap factor k .

Kita tambahkan:

$$P_F (B_k^T x)^2.$$

Karena jika:

$$B_k^T x = 0,$$

penalty:

$$0.$$

Jika exposure besar:

$$(B_k^T x)^2$$

membesar.

C.28. Misalnya kita ingin membatasi BTC beta

Kita tidak harus selalu memaksa:

$$B_{BTC}^T x = 0.$$

Kita bisa memberikan target:

$$B_{BTC}^T x = b^*.$$

Penalty:

$$P_{BTC}(B_{BTC}^T x - b^*)^2.$$

Ini jauh lebih fleksibel.

C.29. Sekarang constraint gross exposure

Misalkan:

$$\sum_i |x_i| \leq K.$$

Untuk binary long-short:

$$|x_i| = y_i^L + y_i^S$$

jika long dan short tidak boleh simultan.

Maka:

$$\sum_i (y_i^L + y_i^S) \leq K.$$

Sekali lagi dapat dipenalti:

$$P_K \left(\sum_i (y_i^L + y_i^S) - K \right)^2.$$

C.30. Sekarang risk budget

Risk portfolio:

$$R(\mathbf{x}) = \mathbf{x}^T \Sigma \mathbf{x}.$$

Jika kita ingin:

$$R(\mathbf{x}) \leq R_{\max},$$

kita bisa menggunakan:

$$P_R [\max(0, \mathbf{x}^T \Sigma \mathbf{x} - R_{\max})]^2.$$

Namun fungsi $\max$ tidak langsung QUBO-friendly.

Untuk versi QUBO, kita lebih baik menggunakan:

quadratic equality dengan slack variables

$$\mathbf{x}^T \Sigma \mathbf{x} + s - R_{\max} = 0.$$

Kemudian:

$$P_R (\mathbf{x}^T \Sigma \mathbf{x} + s - R_{\max})^2.$$

Tetapi ini dapat meningkatkan jumlah qubit secara signifikan.

C.31. Karena itu saya menyarankan pendekatan berbeda untuk versi pertama

Jangan langsung memasukkan semua constraint.

Kita buat tiga tingkat.

C0

$\text{return} + \text{risk}$

C1

Tambahkan:

$\text{trade} - \text{count}$

dan:

$\text{long/short exclusivity}$

C2

Tambahkan:

factor exposure

C3

Tambahkan:

risk budget

dan constraint yang lebih kompleks.

Ini akan membuat debugging jauh lebih mudah.

C.32. Hubungan dengan Max-Cut

Sekarang muncul sesuatu yang menarik.

Risk term:

$$x^T \Sigma x$$

memiliki pairwise interaction:

$$\sum_{ij} x_i x_j.$$

Max-Cut juga memiliki objective pairwise:

$$\sum_{i < j} w_{ij} (1 - x_i x_j)$$

untuk:

$$x_i \in \{-1, +1\}.$$

Artinya kedua masalah memiliki struktur:

pairwise interaction graph.

Untuk trading:

$$\Sigma_{ij}$$

adalah coupling ekonomi.

Untuk Max-Cut:

$$w_{ij}$$

adalah edge weight.

Jadi kita benar-benar sedang mengubah **asset correlation network** menjadi **optimization graph**.

C.33. Interpretasi graph

Anggap setiap aset:

i

sebagai node.

Edge:

(i, j)

memiliki weight:

$$w_{ij} \sim \Sigma_{ij}.$$

Maka:

- positive covariance $\rightarrow$ posisi searah meningkatkan risk,
- negative covariance $\rightarrow$ posisi searah dapat mengurangi risk,
- covariance besar $\rightarrow$ interaction kuat.

Dengan demikian optimizer mencari kombinasi node yang memberikan:

high expected alpha + low interaction risk.

Ini jauh lebih menarik daripada sekadar memilih top- k predicted returns.

C.34. Sekarang TSP juga bisa masuk

TSP tidak cocok sebagai **core asset-selection problem** kita.

Tetapi TSP sangat masuk akal untuk **execution/order sequencing**.

Misalnya sistem ingin melakukan:

N

order.

Ada urutan:

$$i_1 \rightarrow i_2 \rightarrow \dots \rightarrow i_N.$$

Jika latency/slippage bergantung pada urutan eksekusi, kita dapat membuat:

c_{ij}

sebagai cost berpindah dari order i ke j .

Kemudian:

$$\min \sum_{ij} c_{ij} x_{ij}.$$

Ini adalah TSP-like optimization.

Jadi:

Portfolio selection $\rightarrow$ QUBO

sedangkan:

Execution sequencing $\rightarrow$ TSP/QUBO.

Kita tidak perlu memaksakan TSP ke masalah yang sebenarnya bukan TSP.

C.35. Dan Max-Cut?

Max-Cut bisa digunakan untuk:

market-neutral clustering / long-short allocation.

Misalnya kita ingin membagi aset menjadi:

L

dan:

S.

Dengan objective berdasarkan covariance/correlation graph.

Secara konseptual:

positive correlation $\rightarrow$ prefer opposite sides

jika tujuan kita adalah hedging.

Sedangkan:

negative correlation → prefer same side

tergantung definisi objective.

C.36. Jadi kita tidak memiliki "satu quantum algorithm"

Arsitektur kita justru dapat memiliki beberapa optimizer.

Problem 1 — Asset selection

QUBO

Problem 2 — Long/short allocation

QUBO / Max-Cut-like

Problem 3 — Position sizing

continuous / integer optimization

Problem 4 — Execution sequencing

TSP

Problem 5 — Rebalancing

dynamic optimization

Ini jauh lebih realistis.

C.37. Sekarang kita turunkan bentuk QUBO secara eksplisit

Definisikan vector binary:

$$\mathbf{y} = [y_1^L, \dots, y_N^L, y_1^S, \dots, y_N^S]^T.$$

Kita ingin:

$$\min_{\mathbf{y} \in \{0,1\}^{2N}} \mathbf{y}^T \mathbf{Q} \mathbf{y} + \mathbf{c}^T \mathbf{y} + c_0.$$

Kenapa minimization?

Karena QUBO biasanya ditulis:

$$\min .$$

Sedangkan utility kita:

$$\max U.$$

Maka:

$$\min(-U).$$

C.38. Expected-return term

Utility:

$$U_\mu = \sum_i \tilde{\mu}_i (y_i^L - y_i^S).$$

Maka minimization objective:

$$-U_\mu = -\sum_i \tilde{\mu}_i y_i^L + \sum_i \tilde{\mu}_i y_i^S.$$

Jadi linear coefficient:

$$c_i^L = -\tilde{\mu}_i$$

dan:

$$c_i^S = +\tilde{\mu}_i.$$

Ini masuk akal.

Jika:

$$\tilde{\mu}_i > 0,$$

long mendapatkan reward:

$$-\tilde{\mu}_i.$$

Short mendapatkan penalty:

$$+\tilde{\mu}_i.$$

C.39. Risk term

Dengan:

$$x_i = y_i^L - y_i^S,$$

maka:

$$x^T \Sigma x = (y^L - y^S)^T \Sigma (y^L - y^S).$$

Expand:

$$= (y^L)^T \Sigma y^L - (y^L)^T \Sigma y^S - (y^S)^T \Sigma y^L + (y^S)^T \Sigma y^S.$$

Karena:

$$\Sigma = \Sigma^T,$$

maka:

$$x^T \Sigma x = (y^L)^T \Sigma y^L - 2(y^L)^T \Sigma y^S + (y^S)^T \Sigma y^S.$$

Maka risk term:

$$\lambda [(y^L)^T \Sigma y^L - 2(y^L)^T \Sigma y^S + (y^S)^T \Sigma y^S].$$

Ini secara langsung membentuk quadratic matrix Q .

C.40. Tambahkan exclusivity

$$P_{LS} \sum_i y_i^L y_i^S.$$

Sehingga:

$$Q_{L_i, S_i} \leftarrow Q_{L_i, S_i} + P_{LS}.$$

Dengan konvensi QUBO yang konsisten, faktor 2 pada off-diagonal harus ditangani hati-hati ketika mengimplementasikan $y^T Q y$. Ini nanti kita tetapkan sekali dalam kode agar tidak terjadi double-counting.

C.41. Tambahkan maximum number of trades

Constraint:

$$\sum_i (y_i^L + y_i^S) \leq K.$$

Untuk tahap awal, kita dapat menggunakan:

$$P_K \left(\sum_i (y_i^L + y_i^S) - K \right)^2.$$

Expand:

$$= P_K \left[\left(\sum_i z_i \right)^2 - 2K \sum_i z_i + K^2 \right]$$

dengan:

$$z_i = y_i^L + y_i^S.$$

Karena:

$$z_i^2$$

untuk binary variable memiliki sifat:

$$z_i^2 = z_i$$

hanya jika z_i sendiri binary; karena z_i merupakan jumlah dua binary, kita sebaiknya expand langsung dalam y_i^L, y_i^S , bukan memperlakukan z_i sebagai binary.

Ini contoh kecil mengapa **penurunan QUBO harus dilakukan secara hati-hati**, bukan sekadar "lempar constraint ke penalty."

C.42. Dan di sinilah kita mulai mendapatkan Hamiltonian Ising

Setelah QUBO:

$$E(\mathbf{y}) = \mathbf{y}^T Q \mathbf{y} + \mathbf{c}^T \mathbf{y}$$

kita gunakan transformasi:

$$y_i = \frac{1 - Z_i}{2}$$

dengan:

$$Z_i \in \{-1, +1\}.$$

Maka:

$$y_i y_j = \frac{1 - Z_i - Z_j + Z_i Z_j}{4}.$$

Substitusi menghasilkan:

$$H_{\text{Ising}} = C + \sum_i h_i Z_i + \sum_{i < j} J_{ij} Z_i Z_j.$$

Konstanta C tidak mempengaruhi argmin.

Jadi:

$$H = \sum_i h_i Z_i + \sum_{i < j} J_{ij} Z_i Z_j.$$

Dan inilah Hamiltonian yang nantinya dapat diberikan kepada QAOA.

C.43. Ini merupakan hubungan paling penting sejauh ini

Kita sekarang mempunyai:



C.44. Tetapi ada satu masalah besar yang harus kita selesaikan

Kita tidak boleh membuat QUBO yang:

secara matematis bagus tetapi secara finansial tidak bisa dieksekusi.

Contohnya optimizer mengatakan:

LONG BTC

dengan:

$SL = 0.01\%$

dari entry.

Secara matematis risk \$1 dapat dicapai dengan posisi sangat besar.

Tetapi market impact dan spread dapat membuat strategi itu buruk.

Maka Tahap C harus memiliki:

execution feasibility constraints.

C.45. Kita masukkan transaction cost

Misalkan:

C_i^{fixed}

adalah fixed cost.

Dan:

$C_i^{prop}|p_i|$

adalah proportional cost.

Maka:

$$C(\mathbf{p}) = \sum_i C_i^{fixed} z_i + C_i^{prop} |p_i|.$$

Dalam binary approximation:

$$|p_i| \propto y_i^L + y_i^S.$$

Sehingga:

$$C(\mathbf{y}) = \sum_i c_i^{trade} (y_i^L + y_i^S).$$

Ini hanya menambah linear term ke QUBO.

C.46. Slippage

Untuk short-term trading, kita juga dapat menggunakan:

$$C_i^{slippage} = \kappa_i |p_i|^\eta.$$

Misalnya:

$$\eta \approx 1$$

untuk model sederhana.

Atau:

$$\eta > 1$$

untuk market impact nonlinear.

Jika position size binary-fixed:

$$|p_i| = q_i z_i,$$

maka:

$$C_i^{slippage} \approx \kappa_i q_i^\eta z_i$$

yang kembali menjadi linear.

Ini sangat menguntungkan untuk QUBO.

C.47. Jadi objective yang lebih realistis

Sekarang:

$$U = EV - \lambda Risk - TransactionCost - Slippage - ConstraintPenalties.$$

Atau minimization:

$$E = -\text{Expected Value} + \lambda Risk + \text{Cost} + \text{Penalty}.$$

Ini adalah bentuk yang akan kita gunakan.

C.48. C0 — formulasi pertama yang saya sarankan

Untuk menghindari sistem terlalu kompleks, kita mulai dari:

$$\min_{\mathbf{y}} \quad -\tilde{\boldsymbol{\mu}}^T \mathbf{x} + \lambda \mathbf{x}^T \Sigma \mathbf{x} + C(\mathbf{y})$$

subject to:

$$x_i = y_i^L - y_i^S$$

dan:

$$y_i^L + y_i^S \leq 1.$$

Kemudian:

$$\sum_i (y_i^L + y_i^S) \leq K.$$

C.49. Apa yang belum masuk?

Belum:

- dynamic stop loss,
- trailing stop,
- TP adjustment,

- order-book placement,
- execution timing,
- portfolio turnover,
- regime transition,
- liquidity constraints,
- maximum drawdown,
- daily loss limit.

Dan itu memang sengaja.

Karena semua itu sebaiknya datang setelah kita mempunyai core optimizer yang benar.

C.50. Struktur Tahap C lengkap

Saya akan membaginya menjadi:

C1 — Trade candidate generation

Dari:

$$\mu_i, \sigma_i, EV_i$$

tentukan candidate trade.

C2 — Discrete position selection

$$x_i \in \{-1, 0, +1\}.$$

C3 — Position sizing

$$q_i = \frac{R_{\max}}{|P_i - SL_i|}.$$

C4 — Portfolio optimization

$$\max \mu^T x - \lambda x^T \Sigma x.$$

C5 — Factor constraints

$$|B^T x| \leq E_{\max}.$$

C6 — Execution cost

$$C(x).$$

C7 — QUBO

$$\min_x x^T Q x + c^T x.$$

C8 — Ising

$$H = \sum_i h_i Z_i + \sum_{i < j} J_{ij} Z_i Z_j.$$

C.51. Dan satu hal yang sangat penting untuk skripsimu

Ada hubungan langsung dengan apa yang sudah kamu kerjakan sebelumnya.

Kamu sebelumnya telah mempelajari:

Markowitz

QUBO

Ising

VQE

QAOA

dan:

EPG warm-start.

Sekarang semuanya dapat ditempatkan pada satu arsitektur yang jauh lebih besar:

Quantitative Trading

↓

Conditional Factor Model

↓

Dynamic Portfolio Optimization

↓

QUBO

↓

Ising

↓

QAOA

dan kalau kita ingin menggunakan penelitianmu sebelumnya:

EPG Nash Equilibrium → QAOA Initial State / Warm Start.

Jadi penelitian quantum portfolio optimization yang kamu kerjakan bukan sesuatu yang terpisah. Ia dapat menjadi **salah satu optimizer di dalam trading architecture yang lebih besar**.

C.52. Namun saya ingin kita membuat satu keputusan desain yang penting

Jangan membuat QAOA memutuskan semuanya.

Untuk sistem yang realistis, saya menyarankan:

Classical model → candidate generation

Quantum optimizer → discrete combinatorial selection

Classical risk engine → position sizing

Execution engine → order placement

Karena quantum optimization paling masuk akal ketika masalahnya memang **combinatorial**, bukan ketika kita memaksa seluruh sistem menjadi quantum.

C.53. Bentuk akhir Tahap C

Dengan demikian, pada waktu t :

Input

$$B_t = (\mu_t, \Sigma_t, B_t)$$

plus:

$$M_t = (\text{price, spread, liquidity, fees})$$

Candidate generation

$$A_t = \{i : EV_i > EV_{\min}\}$$

Decision

$$y_i^L, y_i^S \in \{0, 1\}$$

Objective

$$E(\mathbf{y}) = -\tilde{\mu}^T \mathbf{x} + \lambda \mathbf{x}^T \Sigma \mathbf{x} + C(\mathbf{y}) + P(\mathbf{y})$$

QUBO

$$E(\mathbf{y}) = \mathbf{y}^T Q \mathbf{y} + \mathbf{c}^T \mathbf{y} + C$$

Ising

$$H = \sum_i h_i Z_i + \sum_{i < j} J_{ij} Z_i Z_j$$

Solver

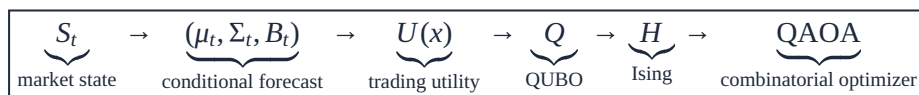
QAOA / classical exact solver / classical heuristic

Output

$$\mathbf{y}^* \rightarrow \{LONG, SHORT, FLAT\}$$

Dan menurutku inilah bagian yang paling penting

Kita sekarang telah berhasil membangun **rantai matematis yang utuh**:



dan yang paling menarik, Σ_t **dari Tahap B menjadi sumber coupling** J_{ij} . Jadi korelasi antar-aset bukan hanya statistik deskriptif; ia secara langsung menentukan interaksi antar-qubit dalam Hamiltonian trading.

Sementara itu, **TSP tidak perlu dipaksakan ke asset selection**. Ia lebih natural ditempatkan nanti pada *execution sequencing*. **Max-Cut** dapat muncul pada long/short partition atau hedging, dan **portfolio optimization** menjadi parent problem yang menghasilkan QUBO kita.

Dengan demikian kita tidak sedang membuat "bot trading yang kebetulan memakai QAOA". Kita sedang membangun:

dynamic stochastic combinatorial optimization framework for systematic trading

yang kebetulan memiliki **quantum optimization layer**.

Dan setelah Tahap C ini, **Tahap D seharusnya bukan langsung coding QAOA**. Tahap D yang paling logis adalah **menurunkan QUBO trading kita secara lengkap dengan contoh numerik kecil**—misalnya 4 aset,

masing-masing LONG/SHORT/FLAT, risk \$1, TP/SL, transaction cost, covariance, dan factor exposure—sampai kita benar-benar mendapatkan matriks Q , lalu mengubahnya menjadi h_i dan J_{ij} . Setelah itu baru kita bisa melihat dengan jelas **apa sebenarnya yang sedang dioptimalkan oleh QAOA** dan berapa qubit yang diperlukan.